ASSIGNMENT 13.5

2303A51356-POOJITHA.EDDE
Batch 20

Task Description #1 (Refactoring – Removing Global Variables)
• Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:
o Refactored version passing rate as a parameter or using a configuration structure.

Prompt:

```
#rate = 0.1
#def calculate_interest(amount):
#return amount * rate
#print(calculate_interest(1000)
#eliminate unnecessary global variables from the above code and make the function more flexible by allowing the caller to specify the interest rate as a parameter.
```

CODE:

```python
#rate = 0.1
#def calculate_interest(amount):
#return amount * rate
#print(calculate_interest(1000)
#eliminate unnecessary global variables from the above code and make the function more flexible by allowing the caller to specify the interest rate as a parameter
def calculate_interest(amount, rate=0.1):
    return amount * rate
# Test cases
print(calculate_interest(1000))  # Using default rate
print(calculate_interest(1000, 0.2))  # Using custom rate
```

OUTPUT:

```python
#rate = 0.1
#def calculate_interest(amount):
#return amount * rate
#print(calculate_interest(1000)
#eliminate unnecessary global variables from the above code and make the function more flexible by allowing the caller to specify the interest rate as a parameter
def calculate_interest(amount, rate=0.1):
    return amount * rate
# Test cases
print(calculate_interest(1000))  # Using default rate
print(calculate_interest(1000, 0.2))  # Using custom rate
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/Assignment-13.5.py"
100.0
200.0
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

OBSERVATION:

The improved calculate_interest function is more adaptable since it lets the user pass the interest rate as a parameter, making it suitable for various use cases. It also avoids using global variables, which improves modularity and minimizes unintended side effects.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Prompt:

```
#score = 78
#if score >= 90:
#print("Excellent")
#else:
#if score >= 75:
#print("Very Good")
#else:
#if score >= 60:
#print("Good")
#else:
#print("Needs Improvement")
#efactor deeply nested if–elif–else logic into a cleaner structure. Readability  Logical simplification
Maintainability Flattened logic using guard clauses or a mapping-based approach.
```

CODE:

```python
#score = 78
#if score >= 90:
#print("Excellent")
#else:
#if score >= 75:
#print("Very Good")
#else:
#if score >= 60:
#print("Good")
#else:
#print("Needs Improvement")
#efactor deeply nested if-elif-else logic into a cleaner structure. Readability  Logical simplification  Maintainability Flattened logic using guard clauses or
def evaluate_performance(score):
    if score >= 90:
        return "Excellent"
    if score >= 75:
        return "Very Good"
    if score >= 60:
        return "Good"
    return "Needs Improvement"
# Test cases
print(evaluate_performance(95))  # Excellent
print(evaluate_performance(80))  # Very Good
print(evaluate_performance(65))  # Good
print(evaluate_performance(50))  # Needs Improvement
```

OUTPUT:

```python
#score = 78
#if score >= 90:
#print("Excellent")
#else:
#if score >= 75:
#print("Very Good")
#else:
#if score >= 60:
#print("Good")
#else:
#print("Needs Improvement")
#efactor deeply nested if-elif-else logic into a cleaner structure. Readability  Logical simplification  Maintainability Flattened logic using guard clauses or
def evaluate_performance(score):
    if score >= 90:
        return "Excellent"
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/.py"
Excellent
Very Good
Good
Needs Improvement
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

OBSERVATION:

The refactored code uses `elif` statements to flatten the nested structure, improving readability and

maintainability. This approach simplifies the logic and makes it easier to understand the flow of conditions at a glance.

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters.

Prompt

```
#f = open("data1.txt")
#print(f.read())
#f.close()
#f = open("data2.txt")
#print(f.read())
#f.close()
```

refactor the repeated file open/read/close logic . focus areas: dry principle,context managers,function reuse

CODE:

```python
def read_file(filename):
    try:
        with open(filename, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: {filename} not found"
# Example usage:
data1 = read_file("data1.txt")
print(data1)
data2 = read_file("data2.txt")
print(data2)
```

OUTPUT:

```python
def read_file(filename):
    try:
        with open(filename, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: {filename} not found"
# Example usage:
data1 = read_file("data1.txt")
print(data1)
data2 = read_file("data2.txt")
print(data2)
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/.py"
Error: data1.txt not found
Error: data2.txt not found
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

OBSERVATION:

The updated code introduces a reusable read_file function that neatly wraps the file reading process, promoting cleaner and more maintainable code. By using a with statement, it automatically handles file closing, which improves safety and prevents resource leaks.

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Prompt:

```
#users = ["admin", "guest", "editor", "viewer"]
#name = input("Enter username: ")
#found = False
#for u in users:
#if u == name:
#found = True
#print("Access Granted" if found else "Access Denied")
#refacor inefficient linear searches using appropriate data structures focus mainly on time complexity and data structure choice
```

CODE:

```python
#users = ["admin", "guest", "editor", "viewer"]
#name = input("Enter username: ")
#found = False
#for u in users:
#if u == name:
#found = True
#print("Access Granted" if found else "Access Denied")
#refacor inefficient linear searches using appropriate data structures focus mainly on time complexity and data structure choice
users = {"admin", "guest", "editor", "viewer"}  # Using a set for O(1) average time complexity
name = input("Enter username: ")
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

OUTPUT:

```python
#found = True
#print("Access Granted" if found else "Access Denied")
#refacor inefficient linear searches using appropriate data structures focus mainly on time complexity and data structure choice
users = {"admin", "guest", "editor", "viewer"}  # Using a set for O(1) average time complexity
name = input("Enter username: ")
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/.py"
Enter username: poojitha edde
Access Denied
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

OBSERVATION:

The improved code utilizes a set for storing users, enabling faster lookups with constant time complexity on average. This optimization replaces slower list searches, making the program more efficient and scalable as the dataset increases.

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt:

```
#salary = 50000
#tax = salary * 0.2
#net = salary - tax
#print(net)

#refactor the above code into class-based design . focus areas: object-oriented design, encapsulation . explected outcome; a class like employeesalarycalculator with methods and attributes
```

CODE:

```python
#salary = 50000
#tax = salary * 0.2
#net = salary - tax
#print(net)

#refactor the above code into class-based design . focus areas: object-oriented design, encapsulation . explected outcome; a class like employeesala
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        return self.salary - tax
# Test case
employee = EmployeeSalaryCalculator(50000)
net_salary = employee.calculate_net_salary()
print(f"Net Salary: {net_salary}")
```

OUTPUT:

```python
#salary = 50000
#tax = salary * 0.2
#net = salary - tax
#print(net)

#refactor the above code into class-based design . focus areas: object-oriented design, encapsulation . explected outcome; a class like employeesala
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/.py"
Net Salary: 40000.0
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

OBSERVATION:

The improved code introduces an EmployeeSalaryCalculator class that groups salary and tax computations together, following object-oriented principles. This approach improves code

organization, enhances readability, and makes future updates or extensions easier to manage.

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

```
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)
```

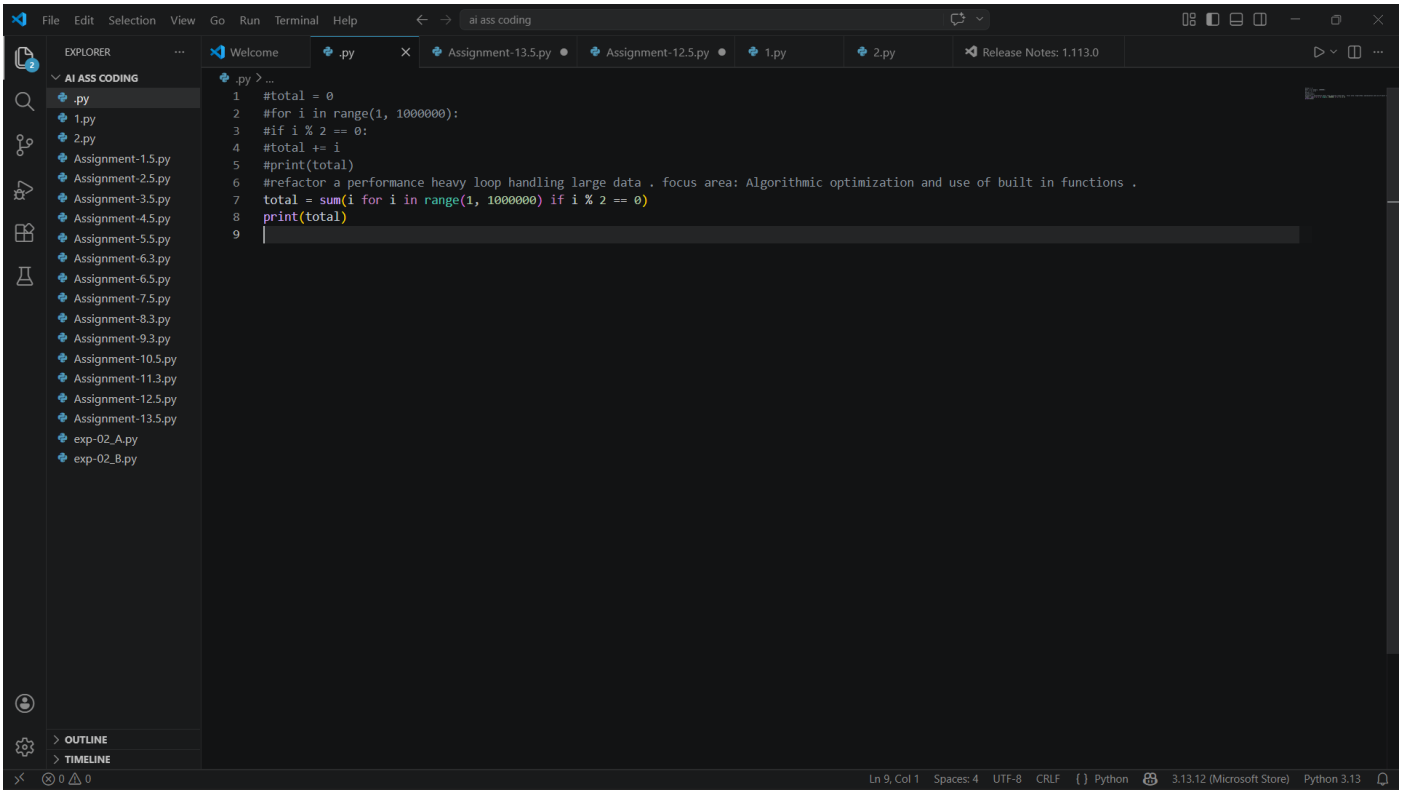
Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

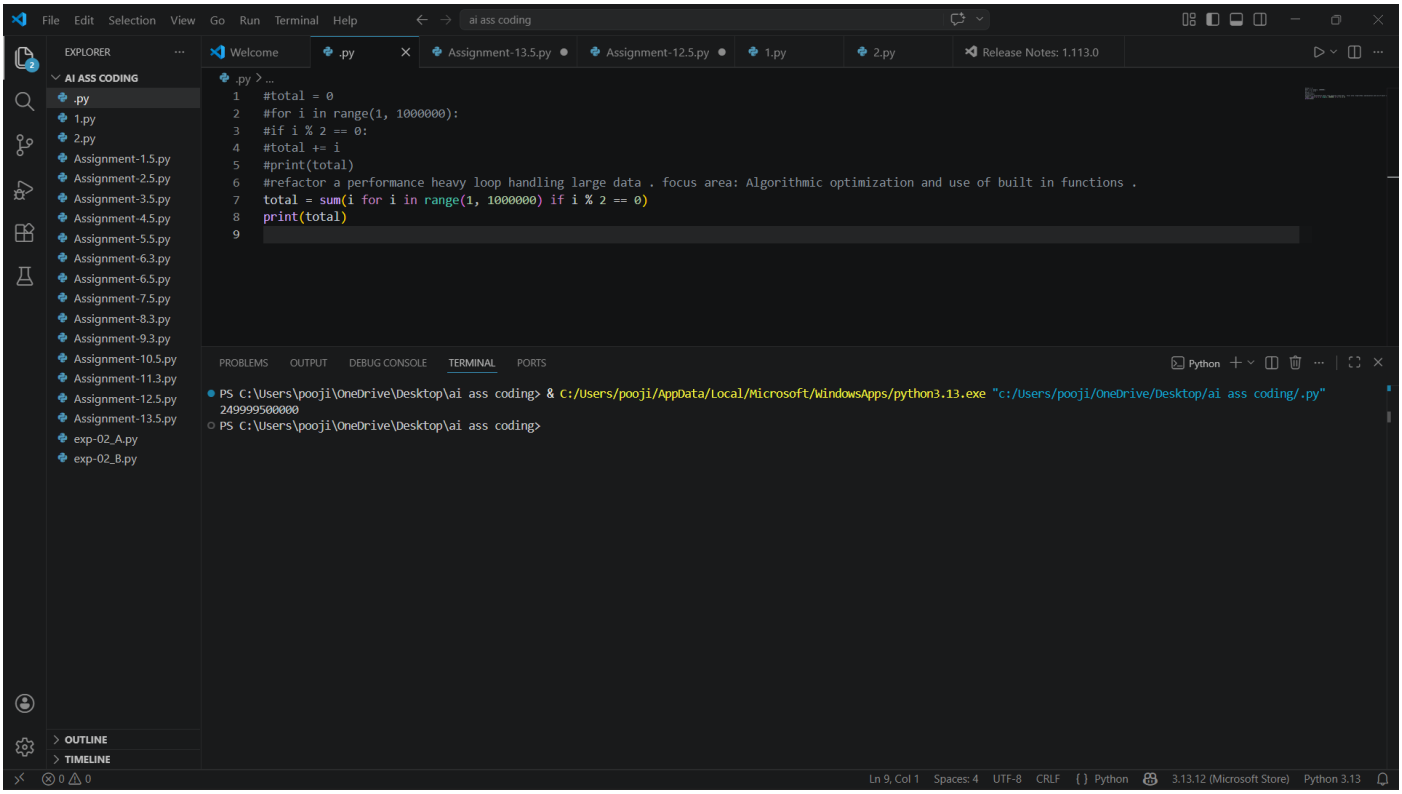
Prompt:

```
#total = 0
#for i in range(1, 1000000):
#if i % 2 == 0:
#total += i
#print(total)
#refactor a performance heavy loop handling large data . focus area: Algorithmic optimization and use of built in functions .
```

CODE:



OUTPUT:



OBSERVATION:

The improved code uses a generator expression inside the sum function, making it more concise and memory-efficient compared to a traditional loop. This approach takes advantage of Python's built-in capabilities, resulting in cleaner code and better performance for larger datasets.

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Prompt:

```
#data = []
#def add_item(x):
#data.append(x)
#add_item(10)
#add_item(20)
#print(data)
#refactor the above code to modify shared mutable state . focus area: functional style refactoring , predictability
```

CODE:

```python
#data = []
#def add_item(x):
#data.append(x)
#add_item(10)
#add_item(20)
#print(data)
#refactor the above code to modify shared mutable state . focus area: functional style refactoring , predictability
def add_item(data, x):
    return data + [x]
# Test cases
data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

OUTPUT:

```python
#data = []
#def add_item(x):
#data.append(x)
#add_item(10)
#add_item(20)
#print(data)
#refactor the above code to modify shared mutable state . focus area: functional style refactoring , predictability
def add_item(data, x):
    return data + [x]
# Test cases
data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/.py"
[10, 20]
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

OBSERVATION:

The improved code returns a new list with the added element instead of altering the original one, avoiding changes to shared mutable data. This functional approach increases reliability, minimizes side effects, and makes the code easier to understand and maintain.

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

Prompt

```
#password = input("Enter password: ")
#if len(password) >= 8:
#if any(c.isdigit() for c in password):
#if any(c.isupper() for c in password):
#print("Valid Password")
#else:
#print("Must contain uppercase")
#else:
#print("Must contain digit")
#else:
#print("Password too short")
#refactor the code to simplify and modularize complex validation rules . focus Area: readability , testability . expected outcome: o Separate validation functions with clear accountability
```

CODE:

```python
#password = input("Enter password: ")
#if len(password) >= 8:
#if any(c.isdigit() for c in password):
#if any(c.isupper() for c in password):
#print("Valid Password")
#else:
#print("Must contain uppercase")
#else:
#print("Must contain digit")
#else:
#print("Password too short")
#refactor the code to simplify and modularize complex validation rules . focus Area: readability , testability . expected outcome: o Separate validation functio
def is_valid_length(password):
    return len(password) >= 8
def contains_digit(password):
    return any(c.isdigit() for c in password)
def contains_uppercase(password):
    return any(c.isupper() for c in password)
def validate_password(password):
    if not is_valid_length(password):
        return "Password too short"
    if not contains_digit(password):
        return "Must contain digit"
    if not contains_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"
# Test cases
print(validate_password("Pass1234"))  # Valid Password
print(validate_password("pass1234"))
print(validate_password("Passwrd"))
print(validate_password("Pass 1234"))
```

OUTPUT:

```python
#password = input("Enter password: ")
#if len(password) >= 8:
#if any(c.isdigit() for c in password):
#if any(c.isupper() for c in password):
#print("Valid Password")
#else:
#print("Must contain uppercase")
#else:
#print("Must contain digit")
#else:
#print("Password too short")
#refactor the code to simplify and modularize complex validation rules . focus Area: readability , testability . expected outcome: o Separate validation functio
def is_valid_length(password):
    return len(password) >= 8
def contains_digit(password):
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/.py"
Valid Password
Must contain uppercase
Password too short
Valid Password
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

OBSERVATION:

The improved code introduces a dedicated is_valid_password function to handle all password checks, making the logic more organized and easier to follow. By separating each validation rule, it enhances readability and simplifies testing and future maintenance.